

Day 43: Create User Class

Day 43 — Create User Class

1. Day Number

Day 43

2. Topic Name

Class and Object

Today you will learn the basic idea of **Object-Oriented Programming (OOP)** using a simple User class.

3. Connection

Earlier, you stored user information using separate variables or dictionaries.

For example, you might have thought about user data like:

```
name = "Rahul"
password = "abc123"
```

or:

```
user = {
    "name": "Rahul",
    "password": "abc123"
}
```

Today, you will learn another way to organize related data:

```
User
├── name
└── password
```

You will create a **User class**, and then create an actual **user object** from that class.

4. Important Topics

Today's important concepts are:

- class
- object
- attributes
- `__init__`
- `self`
- creating an object
- accessing object attributes

The three most important ideas are:

Class	→ blueprint
Object	→ actual thing created from blueprint
Attribute	→ data stored inside the object

5. Foundational Notes

What is a class?

A **class** is a blueprint that describes what an object should contain.

Imagine a form for creating users:

```
User Blueprint

Name: _____
Password: _____
```

The blueprint itself is not a real user.

It only describes what every user should have.

In Python:

```
class User:
    ...
```

User is the name of the class.

By convention, Python class names normally start with a capital letter:

```
User
Product
Transaction
BankAccount
Employee
```

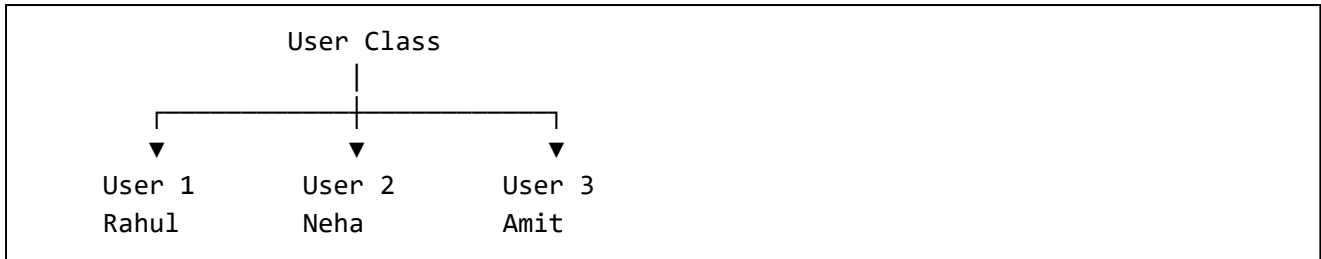
What is an object?

An **object** is an actual value created using a class.

Think about:

```
Class
User
|
| create
▼
Object
Rahul
```

You could create many users from the same User class.



Each object can have its own data.

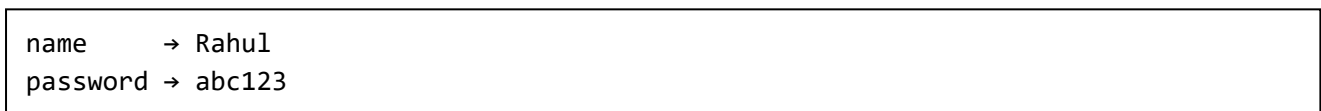
What is an attribute?

An **attribute** is information stored inside an object.

For a user:



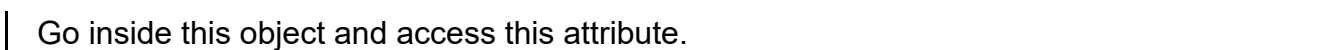
For example:



You normally access an attribute using a dot:



The . means:



What is __init__?

`__init__` is a special method used when an object is created.

Its basic structure looks like:



You can think of `__init__` as the object's **initial setup**.



What is self?

`self` refers to the **current object**.

Suppose two objects exist:

```
User 1
name = Rahul

User 2
name = Neha
```

When Python works with User 1, `self` refers to User 1.

When Python works with User 2, `self` refers to User 2.

So code such as:

```
self.name
```

means:

| Store or access the `name` belonging to this particular object.

This may feel unusual at first. For now, remember:

```
self.attribute
```

means:

```
attribute belonging to this object
```

6. Easy Example

Here is a different example so that we don't solve today's User problem directly.

Suppose we want to represent a book.

Conceptually:

```
class Book:
    def __init__(self, title):
        self.title = title
```

Then an object could be created:

```
book1 = Book("Python Basics")
```

And its title could be accessed:

```
print(book1.title)
```

Output:

Python Basics

The flow is:

```
Book class
  ↓
Book("Python Basics")
  ↓
book1 object
  ↓
title attribute
  ↓
"Python Basics"
```

Notice the difference:

```
Book      → class
book1     → object
title     → attribute
Python Basics → attribute value
```

7. Problem Statement

Create a User class with:

- name
- password

Then:

1. Create one user object.
2. Give the object a name and password.
3. Print only the user's name.

Keep the program very simple.

Example idea:

```
User
├── name
└── password
```

Your program should roughly follow:

```
Create User class
  ↓
Store name and password
  ↓
Create one User object
  ↓
Access object's name
```

↓
Print name

8. Concepts Used

You will practice:

- defining a class
- class keyword
- `__init__`
- `self`
- parameters
- attributes
- creating an object
- accessing attributes with `.`
- `print()`

This is your first step toward OOP concepts that later allow you to build structures such as:

```
User
Product
Cart
Transaction
Order
BankAccount
```

9. Thought Process

Before writing code, break the requirement into small questions.

Step 1: What real-world thing am I representing?

A:

```
User
```

Therefore, you need a class representing a user.

Step 2: What information does a user need?

The problem gives two pieces of data:

```
name
password
```

These should become attributes.

Conceptually:

User Object

```
name      = ?  
password = ?
```

Step 3: When should these values be assigned?

They should be assigned when the user object is created.

That suggests using:

```
__init__
```

Step 4: How does an object keep its own values?

Using:

```
self.name  
self.password
```

Conceptually:

```
self  
↓  
current User object  
├── name  
└── password
```

Step 5: Create an object

After defining the class, create one actual user.

Think:

```
user1 = User(...)
```

Don't copy this blindly. Decide what arguments your constructor requires.

Step 6: Print only the name

You do not need to print the whole object.

Access its name attribute using dot notation.

Think:

```
object.attribute
```

10. Pseudocode

```
START

DEFINE a User class

    DEFINE initialization method
        RECEIVE name
        RECEIVE password

        STORE name in object's name attribute
        STORE password in object's password attribute

CREATE one User object
    PASS a name
    PASS a password

PRINT the object's name attribute

END
```

Another visual version:

```
START
|
v
Define User class
|
v
Define name + password attributes
|
v
Create one User object
|
v
Access object's name
|
v
Print name
|
v
END
```

11. Suggested Solving Approach

Use an **OOP-based approach**.

Your program should have two main parts:

1. Class definition
2. Object usage

Conceptually:

```
class User:
    initialization logic
    ↓
    attributes
        name
        password

create user object
    ↓
print user's name
```

Do not use a dictionary for today's main problem because the purpose is specifically to practice classes and objects.

For example, avoid solving it only like:

```
user = {"name": "...", "password": "..."} 
```

That worked in earlier lessons, but today the goal is:

```
Dictionary approach
    ↓
previous knowledge

Class/Object approach
    ↓
today's new knowledge
```

12. Easy Edge Cases

Edge Case 1: Empty name

Suppose:

```
name = ""
```

Technically, Python can still store it.

Your object might contain:

```
name = ""
password = "abc123"
```

Printing the name would therefore appear blank.

For today's exercise, you don't need advanced validation unless you want additional practice.

Later you could check:

```
if name is empty
    reject it
```

Edge Case 2: Empty password

Suppose:

```
password = ""
```

Again, Python can store an empty string.

For this beginner exercise, understand that:

```
empty string
```

and

```
None
```

are different.

For example:

```
""      → string exists but contains no characters
None    → represents no value
```

Real applications should validate passwords, but that is outside today's simple exercise.

13. Expected Input

You do **not necessarily need** `input()` today.

You can directly provide values while creating your object.

Conceptually:

```
Name: Rahul
Password: pass123
```

The important part is that these two values are passed into your `User` object.

14. Expected Output

Since the problem asks you to print only the name, your output might look like:

```
Rahul
```

You should **not print the password** for this exercise.

Conceptually:

Object

```
| name = Rahul      → print this  
| password = ***** → don't print
```

15. Hint Only

Start with this structure:

```
class User:  
    def __init__(self, ..., ...):  
        # create the two attributes here
```

Then think about:

How do I create an object from User?

How do I access one attribute using a dot?

Remember:

```
Class definition  
  ↓  
Object creation  
  ↓  
Attribute access
```

And the most important relationship for today is:

```
class User  
  ↓  
creates  
  ↓  
user object  
  ↓  
contains  
├── name  
└── password
```

Try writing the solution yourself without using a dictionary.